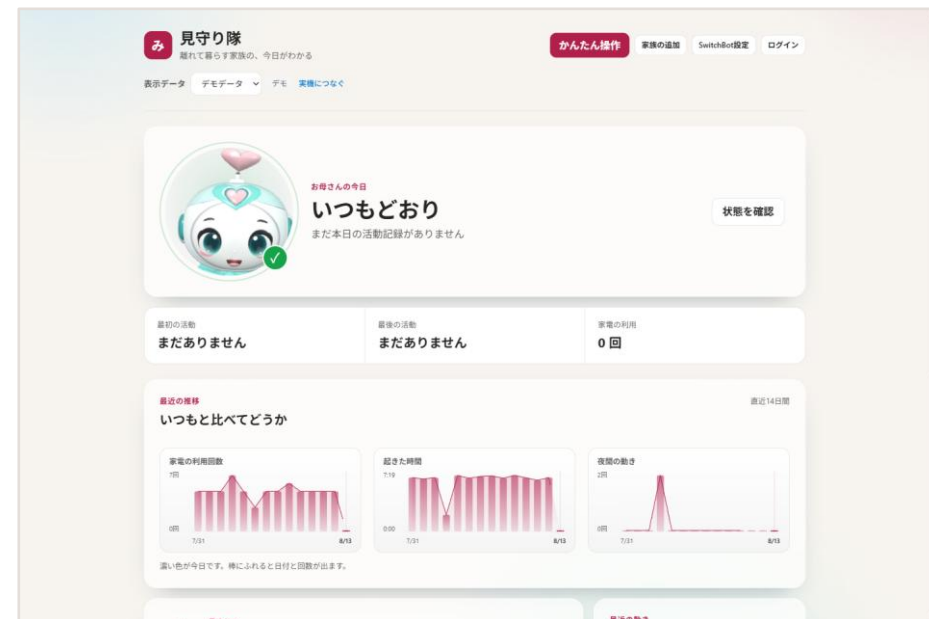


見守り隊

CareRoute AI

見守りサービス

家電の電源で生活リズムを見る



Azure ・ Microsoft Fabric ・ .NET 10 Blazor ・ LINE

まず、動いているところを

説明より先に、実物を見てください



運用コンソール (WebGL)
映っている数字はダミーではなく、その時点の実データ



3Dマスコット「ミマモ」
既製の素材ではなく、自作の MCP サーバーで Blender を操作して制作

通しのデモ動画 (5分51秒・字幕+ナレーション入り)

<docs/demo/mimamoritai-demo.mp4>

公開リポジトリに同梱。本番環境を実際に操作して録画したもの

カメラは1台もない。この画面はすべて、家電の電源が入った記録だけで出来ている

毎日の「元気かな」が、負担になる

見守る側と、見守られる側の両方に

見守る側

毎日電話するのは気が引ける。
かといって、何も知らないのは不安。

見守られる側

カメラを付けられるのは抵抗がある。
監視されている感じがする。

取れる情報を減らして、受け入れやすさを取る

映像も音声も取らない。家電の ON/OFF と消費電力だけを見る。

見られたくない人と、知りたい人。その両方が受け入れられる線を引いた

屋外のデータと、家の中の電力を掛ける

どちらか片方だけでは、危ない瞬間は見つからない

稼働中のシステムが毎日取得している

暑さ指数（WBGT）予測値
環境省 熱中症予防情報サイト

アメダス実況（気温・湿度）
気象庁

天気予報（東京 130000）
気象庁

課題設定の根拠にした東京都オープンデータ

56.6% 熱中症の救急搬送のうち
住居内での発生（令和3年）

65歳以上は中等症以上が半数を超える
搬送は10～14時に集中（在宅の時間帯）
5年推移は減っていない（2023年 7,517人）

環境省の暑さ指数（外は危険） × エアコンが0W（家の中は動いていない） → その重なりだけを家族に知らせる

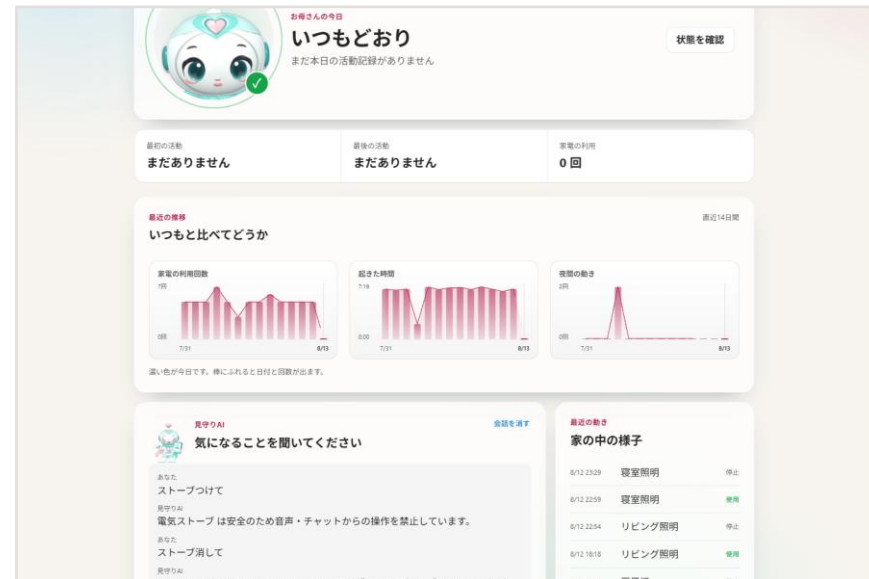
利用データは計10件。出典は画面上にも常時表示しています（環境省 熱中症予防情報サイト／気象庁／東京都オープンデータカタログ CC BY 4.0）

電源の履歴だけで、ここまでわかる

絶対値ではなく「いつもとの差」を見る

朝	6:00 - 9:00 照明が点いた	→ 起きた
昼	9:00 - 18:00 何も動かない	→ いつもと違う
夜	0:00 - 5:00 何度も点く	→ 眠れていないかも

この判定は AI ではなく、ルールベースで行う



直近14日間の推移（濃い色が今日）

しきい値ではなく、その人自身の平常と比べる

なぜ、AI が必要なのか

ルールだけでは「異常」は出せても、「様子」は答えられない

ルールベースで足りる

- 照明が12時間点かない → 異常
- 深夜に3回以上点灯 → 注意
- 安全判定（ON/OFF の可否）

判定は速く、説明でき、外部APIに依存しない。
だからここは AI に渡していない。

ルールベースでは足りない

- 「今日のお母さん、どう？」
- 「最近ちょっと変じゃない？」
- 数値ではなく、言葉で返す

家族が知りたいのは閾値超えの有無ではなく、
「いつもと比べてどうか」という解釈。

AI は「言葉」に使い、「判断」には使わない — 役割を分けたから、両方を捨てずに済んだ

AI に、危険な判断をさせない

このプロジェクトで最も時間をかけた部分

「ストーブつけて」を誤解して、真夏に暖房を入れる。
高齢者の家でそれをやると、便利どころか事故になる。

- 1 **機器を安全クラスで分ける**
照明・扇風機は Safe、暖房・調理器具は Guarded
- 2 **ON と OFF を非対称にする**
ON は安全確認のあと、OFF はそのまま許可
- 3 **拒否も含めて全部記録する**
成功・失敗・拒否をすべて監査ログへ

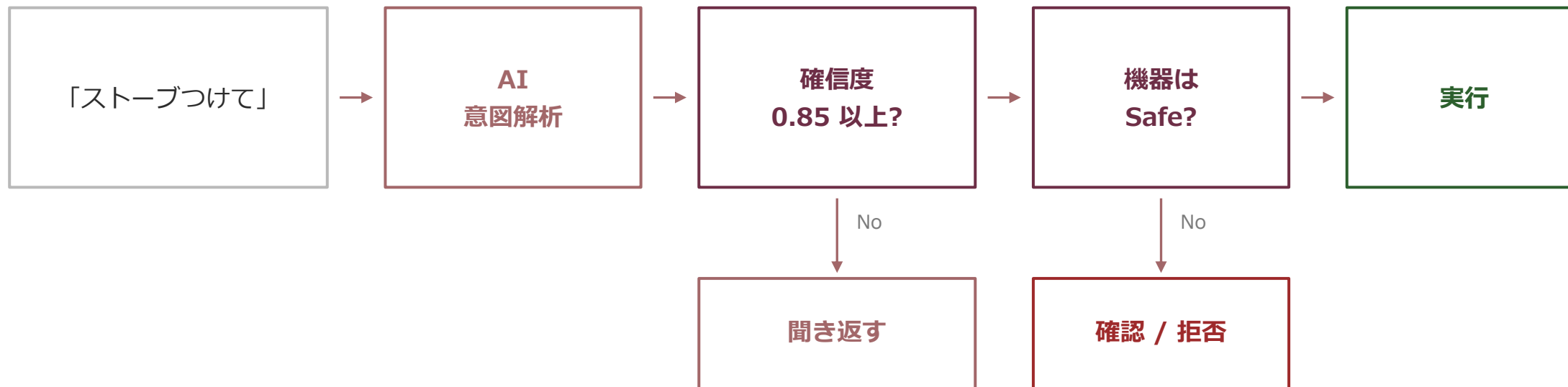


「ストーブつけて」→ まず安全確認。機器は停止中のまま

すぐには通さない。理由を言う。それが家電を任せる条件だった

AI の出力は「候補」でしかない

最終判断は必ずルールベースの層を通る



すべての経路が監査ログへ集まる — 成功も、確認も、拒否も

「消す」は通す

安全性を高める方向の操作まで止めると、使い物にならない

ストーブ つけて

確認が先

火や熱をあつかう機器です。
周囲の安全を確認してから操作します。

ストーブ 消して

そのまま受理

確認をはさまずに実行します。

なぜ今すぐ動かないのかが使う人に見えないと、ただの故障に見える。
そのため機器カードにも「周囲の安全を確認したうえでONにします」と表示している。

止めるのは危険を増やす操作だけ。減らす操作は通す

「鳴らさなかった日」を数えている

見守りの寿命を決めるのは、検知率ではなく誤報率

	通知した	通知しなかった
平常日 通知は不要 ・ 32 件	0 誤報	32
異常日 通知が必要 ・ 20 件	20	0 見逃し

誤報率

0.0 %

平常日 32 件すべてで沈黙。見逃しも 0 件

検知率は「全部に鳴らせば 100%」で作れる。
難しいのは、何でもない日に黙っていること。

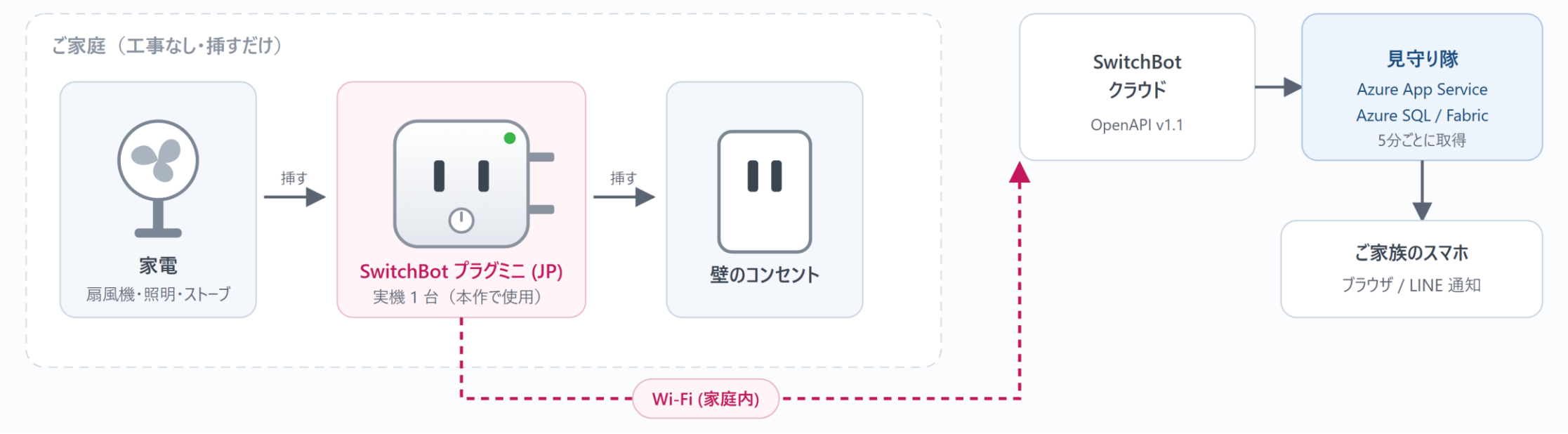
この評価は、最初の実行で自分のバグを 1 件見つけた

「活動量が普段より少ない」の判定が、進行中の一日の集計を、終わった一日の平均と比べていた。
朝は少なくても当たり前なので、全世帯が毎朝鳴る。しかも鳴る引き金は、起きて照明を点けたことだった。
毎朝鳴る通知は読まれなくなり、読まれなくなった通知は、本当の日に効かない。

判定は引数だけで完結する純関数。LLM も外部 API も使わないので、毎 push で CI が再計算する

家に置くのは、コンセント1個だけ

カメラもセンサーも増やさない。工事配線もない

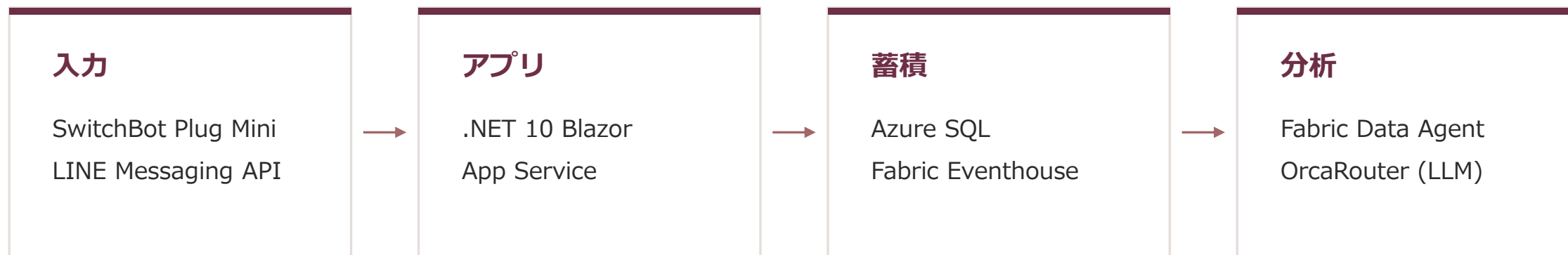


消費電力 動いているか／止まっているか	その日の積算電力量 いつもの一日と重ねて比べる	ON / OFF の時刻 起きた時間・最後に動いた時間	遠隔で ON / OFF 「扇風機をつけて」で操作できる
-------------------------------	-----------------------------------	---------------------------------------	--

実機は SwitchBot プラグミニ (JP) 1台。ハブ経由の赤外線リモコン家電にも対応済み (本作の実機構成には含まない)

構成

dotnetだけで全機能が動く



データを2系統に分けた

DeviceEvents（状態変化）と SwitchBotPlugReadings（電力テレメトリ）は、コードを共有していない。片方の障害がもう片方を巻き込まないため。

壊れる範囲をあらかじめ決めておく。それが分離の目的

秘密を、コードにもアプリ設定にも置かない

持たない・通さない・渡さない・見せない、の4つで分けた

持たない

Key Vault + マネージドID

本番の App Service に秘密情報は1件も無い。繋ぐための資格情報すら存在しない。

通さない

Private Endpoint + VNet 統合

Key Vault と Azure SQL へは Private Endpoint 経由でのみ到達する。

渡さない

Entra External ID / LINE Login

認証は OIDC に委譲しパスワードを預からない。可視範囲は毎回サーバーで判定。

見せない

Data Protection + 署名検証

預かったキーは暗号化して保存し再表示しない。webhook は署名を定数時間で検証。

秘密情報が無くても全機能が動く前提は変えていない。Key Vault へ到達できなくても、アプリは起動を続ける。

秘密を扱う場所を減らす。持たなければ、漏れない

LLM コストは、設計で削る

呼ぶ回数を減らす。これが一番効く

センサー経路

5分ごとのポーリング

0 回

LLM 呼び出し

1世帯あたり 288回/日 の取得と判定。すべてルールベースなので、デバイスが増えても LLM 費用は増えない。

会話経路

家族が話しかけたときだけ

1 回

LLM 呼び出し

意図解析・要約・通知文の生成のみ。費用は「世帯数」ではなく「会話した回数」に比例する。

OrcaRouter に選ばれる。締切のある経路だけ固定する

既定は orcarouter/auto。1回のキーで複数プロバイダを使い分け、選ばれたモデル名を画面に出す。

LINE は 8 秒で打ち切られ、auto は同じ問いで 5.6～51 秒とばらつくため、この経路だけ gpt-4.1-mini に固定（3～5 秒）。

画面側も auto がタイムアウトしたら固定モデルへ退避する（本番実測 2.3 秒）＝速さと費用を両取りする。

API キー0本でも全機能が動く — 審査でも運用でも、課金せずに試せる状態を既定にした

沈黙して壊れた

3件とも、例外は一度も投げていない

存在しないテーブルに投げ続けた

アプリ側だけ実装し、Eventhouse にテーブルを作っていなかった。

1日半、400 を返し続けて容量を焼いた。

宛先が Paused のまま、送信は成功していた

Event Hub は正常に受理し、アプリは「送信済み」を記録。

3日分のデータが静かに消えた。

3Dマスコットが一度も起動していなかった

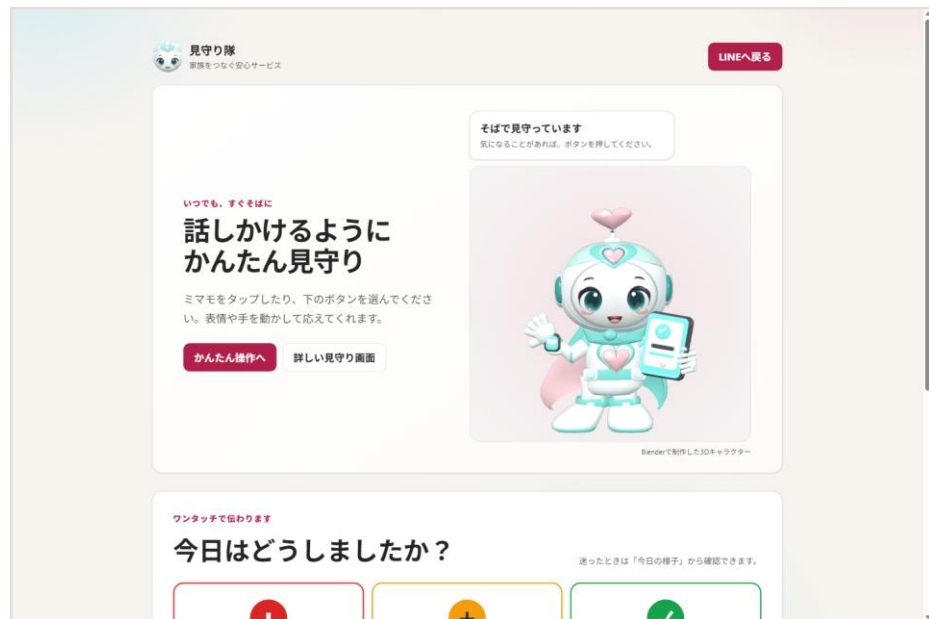
「動いている」と報告したが、実際は初期化されていなかった。

計測方法そのものが間違っていた。

「送信成功」は「到達」ではない。届いた側を見ないと気づけない

見る人と、使う人で画面を分ける

必要な情報がまったく違うため



利用者本人が使う /one-touch 画面

高齢の利用者本人が使う画面

文字とボタンを大きくした専用画面（/one-touch）を用意。
見守る側のダッシュボードとは分けている。

家族は LINE で聞くだけ

専用アプリのインストールは不要。
「今日のお母さんどう？」と聞けば、蓄積された生活データから答える。

見る側と使う側に、同じ画面を出さない

誰が、いくら払うのか

カメラを置けなかった家庭が、そのまま顧客になる

約700万

世帯

65歳以上の一人暮らし
(内閣府 高齢社会白書)

3,000～5,000

円 / 月

既存のセンサー型
見守りサービスの相場

0

円

新規の見守り機器
(家電はすでにある)

成立の条件は「安いこと」ではなく、「置けること」

導入の障壁を外した

カメラもマットも要らない。スマートプラグを既存の家電に挿すだけ。
「見張られる」と感じさせないから、本人が拒まない。

受け手の負担も外した

専用アプリを入れさせない。通知も操作も LINE の中で完結する。
インストール率という最大の離脱要因が、そもそも発生しない。

原価が積み上がらない

判定はルールベース、LLM は会話時のみ。
1世帯あたりの固定費が小さく、月額数百円台でも粗利が残る。

AI は会話に使い、判断には使わない

家電を任せるほど信頼していないから、任せない設計にした

1,065

テスト（全合格）

0

必要な API キー（デモ実行時）

3

記録した「沈黙した障害」

コード	https://github.com/monoqlo78/mimamoritai-tokyo-opendata
デモ動画	リポジトリ内 docs/demo/mimamoritai-demo.mp4（字幕・ナレーション入り / 5分51秒）
解説記事	https://qiita.com/monoqlo78/items/27ea5bfa760bd8e3c3b7

安全判定を *LLM* の外側に置いたか、内側でお願いしたか。そこが一番の分岐点だった。